

SOLARIS

Local AI-Supervised Autonomous Debugging Sandbox

PROJECT DESIGN REPORT · PDR v1.0

TEAM	Team Raftel
INSTITUTION	RV College of Engineering, Karnataka
EVENT	HACKFEST 2026 — NMAM Institute of Technology (Nitte University)
TRACK	Open Innovation
DURATION	36-Hour Hackathon
DATE	April 17, 2026
PLATFORM	Windows Desktop Application (Electron + React + FastAPI)

This document describes the complete system design, architecture, technology stack, team responsibilities, and implementation plan for Solaris — a fully offline, AI-powered autonomous code debugging sandbox packaged as a native desktop application.

■ TABLE OF CONTENTS

1. Problem Statement
2. Solution Overview
3. System Architecture
4. User Flow
5. Technology Stack
6. Key Features
7. Desktop Application Strategy
8. Team Roles & Responsibilities
9. Implementation Timeline (36 Hours)
10. Design Decisions & Trade-offs
11. Risk Analysis
12. Use Cases

1. PROBLEM STATEMENT

Developers spend a disproportionate amount of time debugging code manually. Existing AI-assisted debugging tools introduce two critical problems:

- **Cloud Dependency:** Tools like GitHub Copilot and ChatGPT require live internet access, making them unavailable in offline or air-gapped environments.
- **Privacy & Security Risk:** Proprietary, sensitive, or classified code must be transmitted to external servers — an unacceptable risk for corporate R&D, defense, and financial institutions.
- **No Autonomous Loop:** All existing tools are suggestion-based: they propose a fix, but the developer must manually apply, test, and iterate. No tool closes the feedback loop automatically.
- **No Audit Trail:** There is no existing tool that documents every debugging step, patch attempt, and root cause in a downloadable, traceable report.

2. SOLUTION OVERVIEW

Solaris is a fully offline, AI-powered autonomous debugging system that runs entirely on the user's machine. It accepts broken code, automatically identifies the error, generates a targeted patch using a local LLM, tests the patch in a secure sandbox, and repeats until the code passes — all without any internet connection or cloud dependency. The entire session is documented in a downloadable PDF Audit Trail.

Property	Solaris	Cloud AI Tools
Internet Required	■ Never	■ Always
Code Sent Externally	■ Never	■ Always
Autonomous Fix Loop	■ Yes	■ No
Audit PDF Report	■ Yes	■ No
Air-Gap Compatible	■ Yes	■ No
Sandbox Execution	■ Docker	■ No

3. SYSTEM ARCHITECTURE

Solaris follows a layered pipeline where each component has a single, clear responsibility. The architecture is fully contained within the user's machine.

Layer 1 — Presentation	Electron desktop app wrapping a React frontend. Displays the live iteration log, before/after code diffs, confidence scores, and error classification labels in real time.
Layer 2 — API Gateway	FastAPI backend (Python) serves as the orchestration engine. Receives code from the frontend, manages the repair loop, and coordinates all subsystems.
Layer 3 — Sandbox Execution	Hardened Docker container runs the submitted code in total isolation. Captures stdout, stderr, and exit codes. Flags: <code>--network=none</code> , <code>--memory=256m</code> , <code>--cpus=0.5</code> , <code>--pids-limit=50</code> , with enforced timeout.
Layer 4 — Code Analysis	Python ast module parses the code's structure and classifies the error into one of four categories: Syntactic / Runtime / Logical / Off-by-one. Provides exact node-level location of the fault.
Layer 5 — AI Patch Engine	DeepSeek Coder 6.7B running via Ollama (fully local). Receives error context + original code, returns a targeted line-level patch (not a full rewrite) with a confidence score (1–10).
Layer 6 — Persistence	SQLite stores session history. reportlab generates the downloadable PDF Audit Trail after the session concludes.

4. USER FLOW

01

User opens Solaris desktop app and pastes or uploads broken Python code.

02

React frontend sends code to FastAPI backend via local HTTP (localhost).

03

Backend spawns a hardened Docker container and executes the code.

04

stdout, stderr, and exit code are captured from the container.

05

Python AST engine parses the code and classifies the error type.

06

Error classification + original code is sent to DeepSeek Coder via Ollama.

07

LLM generates a targeted patch and a confidence score (1–10).

08

Patch is applied to the code using difflib.

09

Patched code is re-executed in the Docker sandbox.

10

If code passes → session ends. If it fails → loop returns to Step 03.

11

Loop repeats up to a configurable maximum attempt limit.

12

PDF Audit Trail is generated. Session stored in SQLite history.

5. TECHNOLOGY STACK

Component	Technology	Version / Notes
Desktop Shell	Electron	Latest stable — wraps React as .exe
Frontend UI	React	react-diff-viewer for diffs, live log feed
Backend API	FastAPI (Python)	Uvicorn ASGI server, localhost only
Sandbox	Docker	Hardened: --network=none, --memory=256m, --cpus=0.5
AI Engine	DeepSeek Coder 6.7B	Via Ollama — fully offline, no API key
Code Parser	Python ast module	Built-in, zero dependencies
Diff Engine	Python difflib	Before/after patch comparison
PDF Reports	reportlab	Python library — Audit Trail generation
Session Storage	SQLite	Lightweight local database
Packaging	electron-builder	Produces Windows .exe installer

6. KEY FEATURES



Fully Offline & Air-Gap Safe

Zero cloud dependency. Works in environments with no internet. No API keys needed.



Hardened Docker Sandbox

Code executes in complete isolation. --network=none prevents any outbound calls. Memory, CPU, and PID limits prevent resource exhaustion.



Autonomous Iterative Repair Loop

Automatically re-patches and re-tests until code passes or max attempts is reached. No manual intervention needed.



AST-Based Error Classification

Pinpoints exact code node that failed. Classifies into Syntactic / Runtime / Logical / Off-by-one before LLM patching.



Targeted Patching

LLM produces line-level patches only — preserves developer's original logic and intent. Not a full rewrite.



Live Real-Time UI

[Attempt 1] ■ TypeError → patching... [Attempt 2] ■ Passed — visible in the desktop app as it happens.



Confidence Score

LLM rates its own confidence on each patch (1–10). Displayed alongside each attempt in the UI.



PDF Audit Trail

Downloadable report with every attempt, every patch diff, error classification, and plain-English root cause explanation.



Session History

SQLite stores all past debugging sessions for reference and comparison.



Native Desktop App

Packaged as a Windows .exe via Electron. No browser needed. Launches with a double-click.

7. DESKTOP APPLICATION STRATEGY

Solaris is packaged as a native Windows desktop application using Electron. This decision was made to deliver a polished, double-click-to-run experience without requiring judges or users to manually start a web server.

How Electron integrates with the existing stack:

Electron is a shell that wraps the React frontend in a native OS window. On launch, Electron's main process silently spawns the FastAPI backend (uvicorn) as a child process. The React UI connects to it via localhost. On close, the backend process is killed automatically.

main.js — Electron Entry Point:

```
const { app, BrowserWindow } = require('electron')
const { spawn } = require('child_process')
const path = require('path')

let backendProcess

function createWindow() {
  backendProcess = spawn('python', ['-m', 'uvicorn', 'main:app', '--port', '8000'], {
    cwd: path.join(__dirname, 'backend'),
    windowsHide: true
  })
  const win = new BrowserWindow({ width: 1200, height: 800 })
  setTimeout(() => win.loadURL('http://localhost:3000'), 2000)
}

app.whenReady().then(createWindow)
app.on('window-all-closed', () => {
  if (backendProcess) backendProcess.kill()
  app.quit()
})
```

Build Output:

Step	Command	Output
1. Build React	npm run build	/build folder (static assets)
2. Package .exe	npm run dist	/dist/Solaris Setup.exe

Cross-platform note: Electron supports Windows, macOS, and Linux from the same codebase. Solaris targets Windows .exe for HACKFEST 2026. macOS/Linux builds can be produced with one configuration change.

8. TEAM ROLES & RESPONSIBILITIES

Member 1 — AI Whisperer

- Ollama installation and local model setup
- DeepSeek Coder 6.7B prompt engineering
- LLM API integration (generate targeted patches)
- Iterative repair loop logic
- Confidence score extraction from LLM response

Member 2 — Sandbox Architect

- Docker container setup with hardened security flags
- FastAPI backend and REST endpoint design
- Code execution pipeline (stdin → stdout/stderr capture)
- Timeout enforcement and container lifecycle management
- SQLite session storage integration

Member 3 — Code Analyst

- Python AST parsing engine
- Error type classification (Syntactic / Runtime / Logical / Off-by-one)
- Exact code node fault localization
- difflib-based before/after diff generation
- Patch application logic

Member 4 — Experience Engineer

- React frontend development
- Real-time iteration log feed (WebSocket or polling)
- react-diff-viewer integration for code diffs
- Electron wrapper (main.js + electron-builder packaging)
- PDF Audit Trail generation (reportlab)

9. IMPLEMENTATION TIMELINE (36 Hours)

Hours	Phase	Deliverable	Owner(s)
0 – 2	Setup & Environment	Repos initialized, Ollama + DeepSeek downloaded, Docker tested	All
2 – 8	Core Backend	FastAPI endpoints, Docker sandbox execution, stdout/stderr capture	M2
2 – 8	LLM Integration	Ollama API calls, prompt template, patch extraction, confidence score	M1
8 – 14	AST Engine	Error classification, node localization, difflib patch application	M3
8 – 14	Frontend v1	Code input, iteration log display, basic diff viewer	M4
14 – 20	Repair Loop	Full loop integration: execute → classify → patch → re-execute	M1 + M2
20 – 26	Integration	Frontend connected to backend, live updates working end-to-end	All
26 – 28	PDF Audit Trail	reportlab PDF generation with all attempt data	M4
28 – 32	Electron Wrapper	main.js, backend spawning, electron-builder packaging to .exe	M4
32 – 35	Testing & Polish	End-to-end tests with sample buggy scripts, UI polish	All
35 – 36	Demo Prep	Rehearse demo flow, prepare 3 sample broken code inputs	All

10. DESIGN DECISIONS & TRADE-OFFS

Docker over Firecracker MicroVMs	Rationale: Firecracker requires bare-metal KVM access and kernel image management. Not feasible in 36 hours. Docker with hardened flags provides sufficient isolation for hackathon scope.	Trade-off: Simpler setup, adequate security, minor isolation trade-off
DeepSeek Coder 6.7B over larger models	Rationale: Larger DeepSeek V2 models require 16GB+ VRAM. The 6.7B runs comfortably on mid-range laptops and produces acceptable patch quality for Python debugging.	Trade-off: Speed and RAM efficiency over maximum accuracy
Python ast over Tree-Sitter	Rationale: Tree-Sitter is more powerful and multi-language but requires external setup. Python's built-in ast module works immediately with zero installation overhead.	Trade-off: Zero setup cost, Python-only limitation
Electron over PyInstaller + Tkinter	Rationale: Electron reuses the planned React frontend with no rebuilding. Tkinter would require a full UI rewrite and produce an inferior interface.	Trade-off: Familiar tech stack, larger binary size (~150MB)
reportlab over jsPDF	Rationale: reportlab runs on the Python backend and has access to all session data natively. jsPDF would require serializing all data to the frontend first.	Trade-off: Simpler data access, PDF generated server-side
SQLite over PostgreSQL	Rationale: SQLite is zero-config, file-based, and ships with Python. Perfect for a local desktop application with no concurrent users.	Trade-off: Zero setup, not scalable to multi-user — irrelevant for this use case

11. RISK ANALYSIS

Risk	Likelihood	Impact	Mitigation
LLM returns non-applicable patch format	High	High	Strict prompt template with output format enforcement + regex extraction
Docker cold-start latency slows the loop	Medium	Medium	Reuse a warm container across attempts instead of cold-starting each time
AST parse fails on syntax errors (ast can't be invalid code)	High	Medium	Detect SyntaxError before AST walk; classify as Syntactic and skip AST
DeepSeek Coder too slow on low-RAM laptops	Medium	High	Pre-download model before hackathon; test on target hardware; set max context length
Electron + Python path issues on different machines	Medium	Medium	Use relative paths in main.js; test on clean Windows VM before demo
Time overrun on Electron packaging	Low	Low	Electron is last step; core app works as a web app if packaging fails

12. USE CASES

 Competitive Programmers Fix algorithm bugs during offline practice sessions without cloud tools.	 Corporate R&D; Teams Debug proprietary code in air-gapped environments where cloud AI is prohibited.	 Defense & Government Classified code environments that cannot use any external service.
 CI/CD Pipelines Auto-propose fixes for failing unit tests before a commit is merged.	 Code Review Documentation Auto-generated audit PDF provides a traceable record of all debugging decisions.	 Education Students learn from the step-by-step patch explanation in the Audit Trail PDF.